1. Introduction to React.js

Question 1: What is React.js? How is it different from other JavaScript frameworks and libraries?

→ React.js is a JavaScript library used for building user interfaces, especially single-page applications. It is different from other frameworks because it uses a virtual DOM, component-based structure, and focuses only on the view layer.

Question 2: Explain the core principles of React such as the virtual DOM and component-based architecture.

→ React uses a Virtual DOM to update only changed parts of the UI, improving performance.
→ It follows a component-based architecture, where the UI is divided into reusable components.

Question 3: What are the advantages of using React.js in web development?

→ React is fast, reusable, easy to maintain, SEO-friendly, and has a large community support.

2. JSX (JavaScript XML)

Question 1: What is JSX in React.js? Why is it used?

→ JSX is a syntax that allows writing HTML inside JavaScript. It is used to make UI code simple and readable.

Question 2: How is JSX different from regular JavaScript? Can you write JavaScript inside JSX?

→ JSX looks like HTML but works inside JavaScript. Yes, JavaScript can be written inside JS

Question 3: Discuss the importance of using curly braces {} in JSX expressions.

→ Curly braces {} are used to insert JavaScript expressions inside JSX.

3. Components (Functional & Class Components)

Question 1: What are components in React? Explain the difference between functional components and class components.

→ Components are reusable building blocks of a React UI.
→ Functional components are simple functions.
→ Class components use classes and can have state and lifecycle methods.

Question 2: How do you pass data to a component using props?

→ We pass data to a component by adding attributes, e.g., <Demo name="Bhavika" />. Inside the component, we use props.name.

Question 3: What is the role of render() in class components?

→ render() returns the JSX that displays the UI.

4. Props and State

Question 1: What are props in React.js? How are props different from state?

→ Props are read-only data passed from parent to child.
→ State is changeable data inside a component.

Question 2: Explain the concept of state in React and how it is used to manage component data.

→ State stores changing data inside a component. When state changes, React automatically updates the UI.

Question 3: Why is this.setState() used in class components, and how does it work?

→ this.setState() updates the state in class components and re-renders the component with the new data.

5. Handling Events in React

Question 1: How are events handled in React compared to vanilla JavaScript? Explain the concept of synthetic events.

→ In React, events are handled using camelCase (e.g., onClick) and passed as functions.
→ React uses synthetic events, it'swrapper events that work the same across all browsers.

Question 2: What are some common event handlers in React.js? Provide examples of onClick, onChange, and onSubmit.

→ **onClick** – triggered when a button is clicked.

→ **onChange** – triggered when input value changes.

→ **onSubmit** – triggered when a form is submitted.

Question 3: Why do you need to bind event handlers in class components?

→ Binding is needed to make sure **this** refers to the current component object.
→ Binding ensures **this** refers to the component instance.

6. Conditional Rendering

Question 1: What is conditional rendering in React? How can you conditionally render elements in a React component?

→ Conditional rendering means displaying elements based on a condition (true or false).

Question 2: Explain how if-else, ternary operators, and && (logical AND) are used in JSX for conditional rendering.

→ **if-else**: used outside JSX for complex logic

→ **ternary operator**: used inside JSX for conditions (condition ? true : false)

→ **&& operator**: renders element when condition is true.

7. Lists and Keys

Question 1: How do you render a list of items in React? Why is it important to use keys when rendering lists?

→ Lists are rendered using the map() function.
→ Keys help React identify elements and improve performance.

Question 2: What are keys in React, and what happens if you do not provide a unique key?

→ Keys are unique IDs. Without keys, React gives warnings and causes performance issues.

8. Forms in React

Question 1: How do you handle forms in React? Explain the concept of controlled components.

→ Forms are handled using state. Controlled components use state to manage input values.

Question 2: What is the difference between controlled and uncontrolled components in React?

→ **Controlled**: input value controlled by React state

→ **Uncontrolled**: input value controlled by DOM

9. Lifecycle Methods (Class Components)

Question 1: What are lifecycle methods in React class components? Describe the phases of a component's lifecycle.

→ Lifecycle methods run at different stages of a component.
→ Phases are: Mounting, Updating, and Unmounting.

Question 2: Explain the purpose of componentDidMount(), componentDidUpdate(), and componentWillUnmount().

→ componentDidMount() – runs after component loads
→ componentDidUpdate() – runs after component updates

→ componentWillUnmount() – runs before component is removed

10. Hooks (useState, useEffect, useReducer, useMemo, useRef, useCallback)

Question 1: What are React hooks? How do useState() and useEffect() hooks work in functional components?

→ React Hooks are functions that allow functional components to use state and lifecycle features.

→ useState() manages state values, and useEffect() performs side effects like data fetching.

Question 2: What problems did hooks solve in React development? Why are hooks considered an important addition to React?

→ Hooks removed the need for class components, reduced code complexity, and improved code reuse.

Question 3: What is useReducer ? How we use in react app?

→ useReducer is used to manage complex state logic using actions and reducers, similar to Redux.

Question 4: What is the purpose of useCallback & useMemo Hooks?

→ They are used to improve performance by preventing unnecessary re-rendering.

Question 5: What's the Difference between the useCallback & useMemo Hooks?

→ useCallback memoizes functions.

→ useMemo memoizes values.

Question 6 : What is useRef ? How to work in react app?

→ useRef is used to access DOM elements and store values without re-rendering the component.

11. Routing in React (React Router)

Question 1: What is React Router? How does it handle routing in single-page applications?

→ React Router is a library used for navigation in single-page applications without page reload.

Question 2: Explain the difference between BrowserRouter, Route, Link, and Switch components in React Router.

→ **BrowserRouter:** wraps the application.
→ **Route:** defines path and component.
→ **Link:** navigates without reload.
→ **Switch:** renders the first matching route.

12. React – JSON-server and Firebase Real Time Database

Question 1: What do you mean by RESTful web services?

→ RESTful services allow communication between client and server using HTTP methods.

Question 2: What is Json-Server? How we use in React ?

→ RESTful services allow communication between client and server using HTTP methods.

Question 3: How do you fetch data from a Json-server API in React? Explain the role of fetch() or axios() in making API requests.

→ Data is fetched using fetch() or axios() to make API requests and get responses.

Question 4: What is Firebase? What features does Firebase offer?

→ Firebase is a backend platform by Google offering authentication, real-time database, hosting, and storage.

Question 5: Discuss the importance of handling errors and loading states when working with APIs in React.

→ It improves user experience and prevents application crashes during API failures.

13. Context API

Question 1: What is the Context API in React? How is it used to manage global state across multiple components?

→ Context API is used to manage global state and share data across components without props drilling.

Question 2: Explain how createContext() and useContext() are used in React for sharing state.

→ createContext() creates a context, and useContext() accesses shared data inside components.

14. State Management (Redux, Redux-Toolkit or Recoil)

Question 1: What is Redux, and why is it used in React applications? Explain the core concepts of actions, reducers, and the store.

→ Redux is a state management library.

- **Actions:** describe events
- **Reducers:** update state
- **Store:** holds application state

Question 2: How does Recoil simplify state management in React compared to Redux?

→ Recoil is easier than Redux, requires less boilerplate, and works naturally with React hooks.